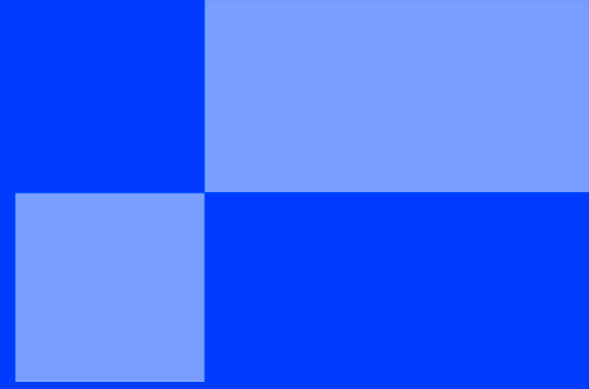
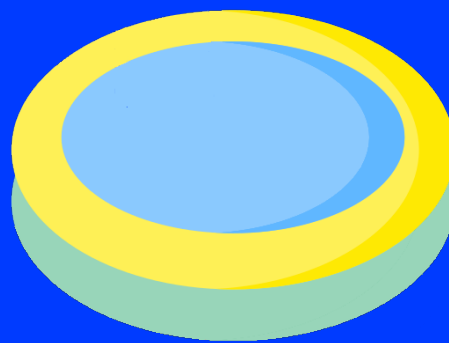




PREPARATION BOOTCAMP

< BONUS TASKS FOR FAST LEARNERS />



PREPARATION BOOTCAMP

Deeper within functions

Task 1.1



Dig this piece of code. Try to figure out its output. Then, run it to see if you were right.

```
crazyFunction = lambda x, y: x * y
meaningOfLife = crazyFunction(6, 7)
print(meaningOfLife)
```

Task 1.2



Use the `sorted()` built-in function to build a new sorted list from a list of lists, but by using the second value (count) inside each secondary list.

Test it with a list such as `animalsCounts = [['cat', 666], ['dog', 3], ['elephant', 42]]`.

Task 1.3



Dig this code `list(filter(lambda x: x > 10, [3.14, 101, 42, 666, -1]))`. Try to predict its output. Then, run it to check if you're right.

Task 1.4



Test this code and try to explain it: `[*enumerate([42, 3, 4, 18, 3, 10])]`

Task 1.5



Create a function `check_even` that returns `True` if a number is even and `False` otherwise. Use your previous function and the `filter()` built-in function in order to output a list of all even numbers contained in `[1, 2, 3, 4, 5, 6]`.

Task 1.6



Use `filter` to remove all strings with more than 4 characters from `['apple', 'banana', 'kiwi', 'pear']`.

Task 1.7



Apply `map` to convert this list of temperatures `[-10, 0, 17.6, 28, 100]`, from Celsius to Fahrenheit.

Task 1.8



Test this code and try to explain it:

```
first_names = ["Jackie", "Chuck", "Arnold", "Sylvester"]
last_names = ["Stallone", "Schwarzenegger", "Norris", "Chan"]
magic = [*zip(first_names, last_names[::-1])]

print(magic[0])
print(magic[1][0])
print(magic[1][1])
```

Task 1.8



Create a function `my_sum` that accepts a variable number of arguments and prints their sum. For instance:

- ✓ `my_sum(1)` returns 1;
- ✓ `my_sum(1,2,3)` returns 6;
- ✓ `my_sum(-20, -10, 5, 5, 10, 10)` returns 0;
- ✓ `my_sum(1, "toto")` throws an *Exception ValueError* but no *Traceback*.

Task 1.9



Write a function `my_division` that takes two integers as parameters, and returns **both** the quotient and the remainder of the euclidean division of the first by the second parameter. It should look like:

```
my_division(42 , 4)
10
2
```



Your function must handle errors.

Task 1.10



Create the function `my_count` that complies with the following conditions:

- ✓ it takes at least one parameter, named `stop`, which is an integer;
- ✓ it can accept a second argument, named `start`, which is also an integer;
- ✓ if the second argument is missing in the function call, assign the **default** value 0 to `start`;
- ✓ it prints all integers from `start` to `stop`.

Task 1.11



Upgrade your previous function with a second default argument, named `step`, which is an integer. If not provided, its default value is `1`. The function prints all integers from `start` (included) to `stop` (excluded).



Your function handles as many cases as possible.
Your function should handle errors.

- ✓ What if `step = 0`?
- ✓ What if `stop < start`?
- ✓ What if `stop = "toto"`?

```
my_count(100, -100, 42)
-100
-58
-16
26
68
```

```
my_count(-100, 100, -42)
100
58
16
-26
-68
```

Task 1.12



Create a function `new_division` that:

- ✓ takes two arguments, `num` and `den`, which are numbers;
- ✓ accepts an optional third argument, `acc`, which is an integer (default = 1);
- ✓ prints the result of the division of `num` by `den` with an accuracy of `acc` digits after the decimal point.



For instance, `new_division(8.4, 13)=0.6` and `new_division(8.4, 13, 6)=0.646154`.

Task 1.13



Create a `ship` function that:

- ✓ takes a variable number of arguments, corresponding to the full name of somebody;
- ✓ takes a variable number of **keyword arguments**, corresponding to the full address;
- ✓ prints the full shipping label in order to mail something to this person.

For instance:

```
ship("Batman", street="Mountain Drive", city="Gotham")  
Batman  
street: Mountain Drive  
city: Gotham
```

```
ship("Superman", "The man of steel", apartment="3D", num= 344, street="Clinton Street",  
city="Metropolis")  
Superman The man of steel  
apartment: 3D  
num: 344  
street: Clinton Street  
city: Metropolis
```



CHALLENGE

Create a program that takes an integer N as an argument and counts how many letters would be used if all the numbers from 1 to N (included) were written out in words.



Spaces and hyphens should NOT be counted.

The use of "and" when writing out numbers is in compliance with British usage.

For instance:

- ✓ 42 (forty-two) contains 8 letters;
- ✓ 115 (one hundred and fifteen) contains 20 letters.

To guide you in your quest, here are some inputs and corresponding outputs:

- ✓ $N = 5$ returns 19 (one, two, three, four, five $\Rightarrow 3 + 3 + 5 + 4 + 4 = 19$);
- ✓ $N = 42$ returns 319;
- ✓ $N = 1000$ returns 21124.



Files manipulation



For the following exercises, you'll need two files:

- ✓ The zen of Python;
- ✓ The table of n , $\text{prime}(n)$ for $n = 1..10000$ from OEIS.org.

Save them in your current working directory with the names `zen.txt` and `primes.txt`.



Your functions should handle errors (non-existing file, unreadable file, empty file, etc.). Before reading a file, you need to open it. After reading a file, you should close it.



Have a closer look at Python **exceptions**.



Task 2.1



Write a `read_file` function that reads and prints **entirely** the file `primes.txt`.

Task 2.2



Write a `read_line` function that reads and prints **line by line** the file `zen.txt`.

Task 2.3



Write a `read_lines` function that:

- ✓ takes a variable number of integers as parameters;
- ✓ then reads and prints the corresponding line(s) from `primes.txt`.

For instance, `read_lines(666)` returns `666 4973` and `read_lines(20000)` throws an error.

Task 2.4



Write a `count_lines` function that reads a file given as parameter, and prints its total number of line(s).



Test your function with both `zen.txt` and `primes.txt` files. To see if it works fine, check each total of lines via a text editor or the `wordcount` command.

Task 2.5



Write a `create_file` function that creates a file named `toto.txt`, in the current directory.

Task 2.6



Write a `write` function that adds the string "I'm a new line" at the end of `toto.txt`.

Task 2.7



Write a `rewrite` function that overwrites the content of `toto.txt` with the content from `zen.txt`.

Task 2.8



Write a `longest` function that prints the longest word contained in `zen.txt`.

Task 2.9

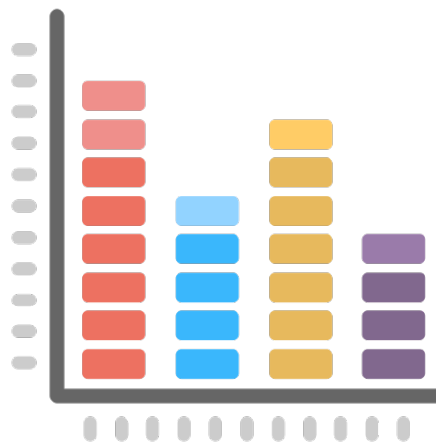


Write a `word_frequency` function that counts the frequency of each word contained in `zen.txt`.

Task 2.10



Write a `letter_frequency` function that counts the frequency of each letter contained in `zen.txt`.



v 0.0.1

{EPITECH}